

PRODUCT REQUIREMENTS DOCUMENT — v2.0

EngPro Production Build

Estimation Ticket Management System — Production Architecture Specification

React + Nginx · NestJS · PostgreSQL

Version 2.0 · 6 August 2026 · Supersedes PRD v1.0 (as-built baseline)

Document control

Item	Detail
Document title	PRD v2.0 — EngPro Production Build Specification
Relationship to v1.0	PRD v1.0 documented the as-built single-file prototype and its findings (SEC-*, DATA-*, BUG-*, GAP-*). This v2.0 defines the production rebuild: same product functionality, re-architected as Frontend (React served by Nginx) + Backend (NestJS) + Database (PostgreSQL). Every v1.0 finding is resolved by design here (Section 12 traceability).
Audience	Development team (FE, BE, DevOps), QA, product owner
Status	Draft for build approval
Naming	Product name standardised as EngPro; ticket prefix standardised as EP- (matching the deployed variant).

Table of contents

Document control	2
Table of contents	2
1. Executive summary — what changes from v1.0.....	4
1.1 Stack decision.....	4
2. Target architecture	6
2.1 Topology	6
2.2 Request path.....	6
2.3 Environments and configuration	6
3. Database design (PostgreSQL 16)	7
3.1 Design principles	7
3.2 Schema (DDL summary).....	7
3.3 Key behaviours in the engine.....	8
3.4 Migration from the legacy Supabase database	8
4. Backend specification (NestJS).....	10
4.1 Module structure	10
4.2 Authentication and session model	10
4.3 RBAC matrix (enforced by RolesGuard + service-level scoping)	10
4.4 REST API catalogue (prefix /api/v1, all JSON, OpenAPI published at /api/docs)	11
4.5 Cross-cutting behaviours	13
5. Frontend specification (React + Nginx)	14
5.1 Application structure	14
5.2 State, data and realtime	14

5.3 UX requirements carried over and added	14
5.4 Nginx responsibilities	15
6. Function-by-function production specification	16
6.1 F1 — Application bootstrap (replaces Supabase config screen)	16
6.2 F2 — Workspace provisioning (replaces setup wizard)	16
6.3 F3 — Authentication & sessions (per-user accounts).....	17
6.4 F4 — Ticket creation (Manager / Admin)	17
6.5 F5 — Ticket creation (Site Admin)	18
6.6 F6 — Dashboard.....	18
6.7 F7 — Kanban board	19
6.8 F8 — Ticket list.....	19
6.9 F9 — Ticket detail & update	20
6.10 F10 — Ticket deletion (soft).....	20
6.11 F11 — Estimator "My tickets"	21
6.12 F12 — Site Admin "My tickets" (repaired).....	21
6.13 F13 — KPI auto-scoring (server-side).....	21
6.14 F14 — KPI dashboard (Manager)	22
6.15 F15 — KPI view (Estimator).....	22
6.16 F16 — Settings & administration	23
6.17 F17 — Realtime synchronisation	23
7. Security requirements (production)	25
8. Non-functional requirements (measurable)	25
9. Deployment & operations	26
9.1 Docker Compose topology.....	26
9.2 CI/CD pipeline (GitHub Actions).....	26
9.3 Monitoring	26
10. Testing strategy.....	26
11. Delivery plan	28
12. Traceability — v1.0 findings resolved	28
13. Scope decisions and open questions	29

1. Executive summary — what changes from v1.0

The prototype proved the product: real teams run 100+ live tenders through it daily. It also proved unshippable as-is: all logic and secrets live in the browser, the database is world-writable, and identity is a shared password. v2.0 keeps the validated workflows and screens and moves every trust decision to a server.

Dimension	v1.0 prototype	v2.0 production
Architecture	Single HTML file → Supabase REST directly from browser	React SPA (Nginx) → NestJS REST + WebSocket API → PostgreSQL
Identity	3 shared role passwords, plain text, compared in browser	One account per person; Argon2id-hashed passwords; JWT access + rotating refresh tokens; server-side RBAC guards
Authorisation	Cosmetic (UI-only); DB fully open via anon key	Enforced per endpoint and per row on the server; DB reachable only from the API
Ticket numbering	Client count+1 (duplicates, reuse)	Atomic per-workspace sequence in PostgreSQL (race-free, never reused)
People linkage	Name strings, best-effort rename cascades	Foreign keys (assignee_id, site_id); renames are metadata only
KPI scoring	Client-side, silent failures, double-award race	Server-side transactional service + DB unique constraint; reversal entries on reopen
Sync	8-second full re-fetch polling per client	WebSocket events per workspace; targeted cache invalidation
Safety	Stored-XSS possible; no audit; hard deletes	React-escaped rendering + server validation; audit log; soft delete
Operations	Netlify static page; no repo evident; no tests	Dockerised 3-tier deployment behind Nginx; Git + CI/CD; automated tests; backups & monitoring

1.1 Stack decision

Layer	Choice	Rationale
Database	PostgreSQL 16	Already the engine underneath the prototype (Supabase), so data migrates natively. Sequences, enums, partial unique indexes, JSONB audit payloads and row-level constraints solve v1.0's integrity findings in the engine itself.
Backend	NestJS 10 (Node 20 LTS, TypeScript) + Prisma ORM	Chosen over Next.js: the frontend is a separate React SPA behind Nginx, so a dedicated API framework fits better than a React meta-framework. NestJS provides modules, dependency injection, guards/interceptors for RBAC, class-validator DTOs, OpenAPI generation and a first-class WebSocket gateway. (Next.js would be the right choice only if we wanted one combined rendering+API app — we do not.)
Frontend	React 18 + TypeScript + Vite, served as static files by Nginx	Preserves the SPA feel of the prototype; React's default JSX escaping closes stored-XSS by construction. Nginx also terminates TLS and reverse-proxies /api and /ws to NestJS.

Layer	Choice	Rationale
FE libraries	React Router 6, TanStack Query 5, socket.io-client, dnd-kit, react-hook-form + zod, i18next	Server-state caching + invalidation pairs naturally with WebSocket events; dnd-kit gives the board drag-and-drop (GAP-3); i18next covers EN/ID (GAP-9).
Infra	Docker Compose (nginx, api, postgres) + GitHub Actions CI/CD	Reproducible builds; single-VM deployment initially, portable to k8s later.

2. Target architecture

2.1 Topology

1. Browser loads the React SPA (static HTML/JS/CSS) from Nginx over HTTPS.
2. The SPA calls the API at `/api/v1/*` ; Nginx reverse-proxies these requests to the NestJS container (no direct exposure of Node).
3. Realtime updates flow over a WebSocket at `/ws` (socket.io), also proxied by Nginx, with connections authenticated by JWT and joined to a per-workspace room.
4. NestJS is the only component with database credentials; it talks to PostgreSQL on the private Docker network.
5. Daily `pg_dump` backups are written to mounted storage; `/api/health` and `/api/ready` endpoints feed uptime monitoring.

2.2 Request path

```
[Browser SPA] --HTTPS--> [Nginx :443]
|-- /                -> static React build (immutable assets, cache-busted)
|-- /api/v1/*        -> proxy_pass http://api:3000 (rate-limited, gzip)
|-- /ws              -> proxy_pass + Upgrade headers (socket.io)
[NestJS :3000] --TCP (private network)--> [PostgreSQL :5432]
```

2.3 Environments and configuration

Environment	Purpose	Notes
dev	Local development	docker compose up; Vite dev server proxies <code>/api</code> ; seeded demo data.
staging	UAT and migration rehearsal	Mirrors production topology; anonymised copy of production data for testing.
production	Live system	TLS (Let's Encrypt), backups, monitoring, restricted SSH.

- All secrets (`DATABASE_URL`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `CORS_ORIGIN`, SMTP creds) come from environment variables / a `.env` file excluded from Git — nothing is ever hard-coded in client or server source (resolves SEC-1).
- CORS locked to the SPA origin; Helmet security headers on the API; Nginx adds HSTS, X-Content-Type-Options, frame-ancestors none, and a Content-Security-Policy for the SPA.

3. Database design (PostgreSQL 16)

3.1 Design principles

- Every relationship is a foreign key with an explicit ON DELETE rule — no name-string linkage anywhere (resolves DATA-2).
- Enumerations are PostgreSQL enum types mirroring the fixed vocabularies validated in v1.0 (5 statuses, 4 work types, 3 priorities, 3 sources).
- Ticket numbers come from an atomic per-workspace counter incremented inside the insert transaction — unique, gapless under normal operation, never reused after deletion (resolves DATA-1).
- Soft delete (deleted_at) on tickets and users; hard delete reserved for GDPR-style purge jobs (resolves GAP-8 with the audit log).
- Single source of schema truth: versioned Prisma migrations in Git (resolves DATA-5).

3.2 Schema (DDL summary)

```
CREATE TYPE user_role AS ENUM ('MANAGER', 'ADMIN', 'SITE_ADMIN', 'ESTIMATOR');
CREATE TYPE ticket_status AS ENUM
('NEW', 'IN_PROGRESS_ESTIMATION', 'IN_PROGRESS_TENDER', 'DONE', 'HOLD');
CREATE TYPE ticket_priority AS ENUM ('URGENT', 'NORMAL', 'LOW');
CREATE TYPE work_type AS ENUM
('PROJECT_TENDER', 'OPS_TENDER', 'SITE_INSTRUCTION', 'BUDGETING_INTERNAL');
CREATE TYPE request_source AS ENUM ('WHATSAPP', 'EMAIL', 'VERBAL');
CREATE TYPE kpi_entry_type AS ENUM ('OPENING', 'AUTO', 'MANUAL', 'REVERSAL');

workspaces (id uuid PK, company, subtitle, ticket_prefix DEFAULT 'EP',
            ticket_seq bigint NOT NULL DEFAULT 0, created_at, updated_at)
sites (id uuid PK, workspace_id FK, name, color int, is_active bool,
        UNIQUE (workspace_id, name))
users (id uuid PK, workspace_id FK, username citext, full_name, email,
        password_hash, role user_role, site_id FK sites NULL, -- required for
SITE_ADMIN
        avatar_color int, is_active bool, must_change_password bool,
        created_at, updated_at, deleted_at NULL,
        UNIQUE (workspace_id, username))
tickets (id uuid PK, workspace_id FK, ticket_no text,
        name, type work_type, priority ticket_priority, status ticket_status,
        requestor, source request_source NULL, description,
        deadline date NULL, assignee_id FK users NULL ON DELETE SET NULL,
        site_id FK sites NULL, submitted_by_id FK users,
        version int NOT NULL DEFAULT 1, -- optimistic locking
        created_at, updated_at, completed_at NULL, deleted_at NULL,
        UNIQUE (workspace_id, ticket_no))
ticket_notes (id uuid PK, ticket_id FK ON DELETE CASCADE, author_id FK users,
        content, status_at_time ticket_status, created_at)
kpi_settings (workspace_id PK/FK, point_opening int DEFAULT 10,
        point_on_target int DEFAULT 3, point_miss_target int DEFAULT -2,
        updated_by_id FK users, updated_at)
kpi_entries (id uuid PK, workspace_id FK, user_id FK users, month int, year int,
        points int, type kpi_entry_type, description,
        ticket_id FK tickets NULL ON DELETE SET NULL, ticket_no text NULL,
        created_by_id FK users NULL, created_at)
-- one auto award per ticket, enforced by the ENGINE, not the client:
CREATE UNIQUE INDEX ux_kpi_auto_once ON kpi_entries (ticket_id) WHERE type = 'AUTO';
-- one opening per member per month:
CREATE UNIQUE INDEX ux_kpi_opening ON kpi_entries (user_id, year, month) WHERE type =
'OPENING';
audit_log (id bigserial PK, workspace_id, actor_id FK users NULL,
```

```

entity_type text, entity_id uuid, action text,
before jsonb NULL, after jsonb NULL, ip inet NULL, created_at)
refresh_tokens (id uuid PK, user_id FK, token_hash, expires_at, revoked_at NULL,
created_at, user_agent text NULL)

```

3.3 Key behaviours in the engine

Behaviour	Mechanism
Ticket number generation	Inside the create-ticket transaction: UPDATE workspaces SET ticket_seq = ticket_seq + 1 WHERE id = \$ws RETURNING ticket_seq; format as <prefix>-NNN (zero-padded, width grows past 999). Row-level lock makes concurrent submissions serialise.
One KPI auto-award per ticket	Partial unique index ux_kpi_auto_once; a concurrent duplicate insert fails with 23505 and is swallowed as "already awarded" (resolves DATA-4).
Reopen reversal	When status leaves DONE, the service inserts a REVERSAL entry with points = -(auto entry points) in the current month, description "Reversal — reopened <ticket_no>". A later completion writes a fresh AUTO award (allowed because the reversal neutralised the ledger; the unique index applies to AUTO rows only — implemented as: AUTO award allowed if sum of AUTO+REVERSAL for the ticket is zero). (Resolves GAP-6.)
Optimistic locking	PATCH must send the version it read; UPDATE ... WHERE id=\$1 AND version=\$2 sets version = version+1; zero rows updated → HTTP 409 Conflict (resolves DATA-6).
Overdue definition	Computed in SQL: deadline < CURRENT_DATE AND status <> 'DONE' AND deleted_at IS NULL — one definition reused by list filters and dashboard stats. Server time (UTC + workspace timezone Asia/Jakarta) is authoritative, never the client clock.
Indexes	tickets: (workspace_id, status), (workspace_id, assignee_id), (workspace_id, deadline), GIN trigram on name for search; kpi_entries: (workspace_id, year, month), (user_id, year); audit_log: (entity_type, entity_id).

3.4 Migration from the legacy Supabase database

1. Export legacy tables as CSV from Supabase (workspaces, members, site_admins, tickets, notes, kpi_settings, kpi_entries, tender_notes).
2. Create the target schema via Prisma migrations; insert the workspace row (prefix EP).
3. members → users: role admin→ADMIN, member→ESTIMATOR; generate username from name (lowercase, deduplicated); set must_change_password = true with a generated temporary password delivered out-of-band. Legacy plain-text passwords are NOT imported.
4. site_admins → sites (one row each) + one SITE_ADMIN user per site (username from site name; temporary password).
5. tickets: map name-string assignee → users.id, site_name/submitted_by → sites.id / users.id (report of unmatched names for manual resolution before cutover); keep legacy ticket_no verbatim; set ticket_seq to the highest imported number so new tickets continue the series; set completed_at = updated_at for DONE tickets.
6. notes → ticket_notes (author matched to users by name; unmatched become a "Legacy import" system user); kpi_entries: member_name → user_id, preserving month/year/points/type and ticket references.
7. tender_notes: schema to be confirmed against the deployed build (Open Question Q1); migrated with the same author/ticket mapping once confirmed.

8. Validation gate: row counts match; sum of KPI points per member per month identical before/after; every non-deleted ticket has a valid workspace, status and unique ticket_no. Rehearse fully on staging before production cutover.

4. Backend specification (NestJS)

4.1 Module structure

Module	Responsibility
AuthModule	Login, refresh-token rotation, logout, password change/reset; JwtAuthGuard + RolesGuard; Argon2id hashing; login rate limiting (5/min/IP + username backoff).
UsersModule	User CRUD (manager only), profile (me), activation/deactivation, avatar colour.
SitesModule	Site CRUD; site ↔ site-admin user pairing.
WorkspaceModule	Workspace identity (company, subtitle, prefix), KPI settings ownership.
TicketsModule	Ticket CRUD, numbering, status transitions, assignment, deadline changes, stats aggregation, filtering/pagination/search.
NotesModule	Progress notes (create, list); immutable once written.
KpiModule	Auto-award + reversal service (invoked by TicketsModule inside the same transaction), opening points, manual bonus/penalty, summaries, export.
AuditModule	Write-behind audit interceptor (entity, action, before/after JSONB, actor, IP); query endpoint for managers.
EventsModule	socket.io gateway: JWT handshake, workspace rooms, emits ticket.*, note.created, kpi.updated, user.updated events after commit.
NotificationsModule (Phase 2)	E-mail on assignment / T-2 deadline / overdue digest; WhatsApp provider integration later (GAP-4).

4.2 Authentication and session model

1. POST /auth/login with username + password. On success: access JWT (15 min; claims sub, workspaceId, role, siteId) + refresh token (7 days, httpOnly Secure SameSite=Strict cookie; hash stored server-side).
2. SPA keeps the access token in memory only; a 401 triggers a silent POST /auth/refresh which rotates the refresh token (old one revoked — replay of a stale token revokes the whole chain).
3. Logout revokes the refresh token server-side and clears the cookie.
4. Users flagged must_change_password are forced through POST /auth/change-password before any other endpoint (enforced by guard).
5. Password policy: minimum 10 characters, checked against a common-password list; managers can trigger reset links / temporary passwords but can never view a password (resolves SEC-3, SEC-4).

4.3 RBAC matrix (enforced by RolesGuard + service-level scoping)

Capability	MANAGER	ADMIN	SITE_ADMIN	ESTIMATOR
Create ticket	Yes	Yes	Yes (own site; assignee not settable)	No
View tickets	All	All	Own site only (server-filtered)	All (board read-only) + own list
Assign / reassign	Yes	Yes	No	No

Capability	MANAGER	ADMIN	SITE_ADMIN	ESTIMATOR
Change status	Any ticket	Any ticket	No	Only tickets assigned to them
Change deadline	Yes	Yes	Own-site tickets only	No
Add note	Yes	Yes	Own-site tickets	Own tickets
Delete (soft) ticket	Yes	Yes	Own-site, only while status = NEW	No
KPI settings / opening / bonus	Yes	No	No	No
KPI visibility	All members	No	No	Own only (any year)
Users / sites / workspace admin	Yes	No	No	No
Audit log	Yes	No	No	No

Note: v1.0 gave Site Admins unconditional delete on their tickets and gave Admins full delete; v2.0 narrows Site Admin deletion to still-untriaged (NEW) tickets — once estimation work has started, withdrawal happens by a note + Hold status. This is a deliberate behaviour change; flagged in Section 13.

4.4 REST API catalogue (prefix `/api/v1`, all JSON, OpenAPI published at `/api/docs`)

Method & path	Roles	Purpose / notes
POST <code>/auth/login</code>	public	Body {username, password} → {accessToken, user}; sets refresh cookie. 429 on rate limit.
POST <code>/auth/refresh</code>	cookie	Rotates refresh token → new access token.
POST <code>/auth/logout</code>	any	Revokes refresh token.
POST <code>/auth/change-password</code>	any	{currentPassword, newPassword}.
GET <code>/me</code>	any	Profile + role + workspace branding for app shell.
GET <code>/workspace</code>	any	Company, subtitle, prefix.
PATCH <code>/workspace</code>	MANAGER	Update identity fields.
GET <code>/users?role=&active=</code>	MANAGER (full), others (id+name+color directory)	Directory powers assignee dropdowns and avatars.
POST <code>/users</code>	MANAGER	Create user with role, optional site, temp password (returned once).
PATCH <code>/users/:id</code>	MANAGER	Rename, role, colour, site, activate/deactivate. Rename never touches tickets (FK linkage).
POST <code>/users/:id/reset-password</code>	MANAGER	Issues temp password, sets must_change_password.

Method & path	Roles	Purpose / notes
DELETE /users/:id	MANAGER	Soft-deactivate; tickets keep assignee_id, UI shows "(inactive)"; option to bulk-unassign.
GET/POST/PATCH/DELETE /sites...	MANAGER	Site CRUD; delete blocked while active tickets reference the site (409).
GET /tickets	per RBAC	Query: page, pageSize (≤100), search (name/ticket_no/requestor, trigram), status, type, priority, assigneeId unassigned, siteId, overdue, sort (createdAt deadline priority ±). Returns {items, total, page}. SITE_ADMIN force-filtered to own site.
POST /tickets	MANAGER, ADMIN, SITE_ADMIN	Creates with server-generated ticket_no; DTO validation (name 3–200 chars, enum fields, deadline not in past unless flag).
GET /tickets/:id	per RBAC	Full detail incl. notes (paged) and KPI award status.
PATCH /tickets/:id	per RBAC (field-level)	Body includes version (409 on mismatch). Field permissions per RBAC matrix. DONE transition triggers KPI award in-transaction; leaving DONE triggers reversal.
DELETE /tickets/:id	MANAGER, ADMIN; SITE_ADMIN (NEW only)	Soft delete + audit entry.
GET /tickets/stats	MANAGER, ADMIN, ESTIMATOR	Dashboard aggregates: totals, by status, by type, overdue, unassigned, per-estimator workload — one SQL round trip.
GET /tickets/:id/notes • POST /tickets/:id/notes	per RBAC	Notes immutable; content 1–2000 chars; status_at_time stamped server-side.
GET /kpi/settings • PUT /kpi/settings	MANAGER (read: any role)	Scoring parameters.
GET /kpi/summary?year=&month=	MANAGER; ESTIMATOR (self only)	Per-member totals + opening/auto/manual/reversal breakdown; month strip totals.
GET /kpi/entries?year=&month=&userId=	MANAGER; ESTIMATOR (self)	Ledger, paged. Any historical year (resolves GAP-7).
POST /kpi/opening	MANAGER	Bulk {year, month, items:[{userId, points}]} — upsert semantics, single transaction.
POST /kpi/entries	MANAGER	Manual bonus/penalty {userId, year, month, points≠0, description required}.
GET /kpi/export?year=	MANAGER	CSV/XLSX ledger export (resolves GAP-6/15).
GET /audit?entityType=&entityId=&page=	MANAGER	Audit trail query.

Method & path	Roles	Purpose / notes
GET /health • GET /ready	public	Liveness / readiness (DB ping).

4.5 Cross-cutting behaviours

- Validation: every DTO validated with class-validator; unknown fields stripped; errors return RFC-7807-style {statusCode, message[], error}.
- Errors: 400 validation, 401 unauthenticated, 403 role/scope, 404 not found in workspace, 409 version conflict / duplicate constraint, 429 rate limited.
- Audit interceptor records every mutating call with before/after snapshots (resolves GAP-8).
- WebSocket events after commit: ticket.created, ticket.updated {id, changedFields}, ticket.deleted, note.created, kpi.updated {userId, year, month}, user.updated. Payloads carry ids, not full rows — clients re-query through TanStack Query invalidation.
- All timestamps stored UTC; API returns ISO-8601; the workspace display timezone (Asia/Jakarta) is applied in the frontend; deadline comparisons use workspace-local dates server-side.

5. Frontend specification (React + Nginx)

5.1 Application structure

Route	Roles	Screen (v1.0 equivalent)
/login	public	Username + password (replaces role-picker; role comes from the account). Force change-password screen when flagged.
/dashboard	MANAGER, ADMIN	Stats, status/type breakdown, workload, recent tickets (F6).
/board	MANAGER, ADMIN, ESTIMATOR	Kanban with filters + drag-and-drop (F7).
/tickets	MANAGER, ADMIN	Paginated, searchable, sortable table (F8).
/tickets/new	MANAGER, ADMIN, SITE_ADMIN	Intake form (F4/F5); site admins get the site-stamped variant.
/tickets/:id	per RBAC	Ticket detail as routed page with modal-style layout: metadata, permitted edit form, notes timeline (F9), soft-delete (F10).
/my-tickets	ESTIMATOR	Personal queue with urgent/overdue sections (F11).
/site/my-tickets	SITE_ADMIN	Site-scoped list, now with working row click → detail incl. deadline edit (fixes BUG-2) (F12).
/kpi	MANAGER	Leaderboard, month strip, settings, opening, bonus modals (F14).
/kpi/me	ESTIMATOR	Personal KPI with year selector (F15).
/settings/*	MANAGER	Workspace · Users · Sites · Audit tabs (F16).

5.2 State, data and realtime

1. TanStack Query owns all server state: queries keyed by endpoint + filters; mutations invalidate affected keys.
2. A socket.io connection (established after login, re-authenticated on token refresh) listens for workspace events and invalidates the matching query keys — the board updates within ~1 s of any colleague's change, replacing v1.0's 8 s polling (F17).
3. Connection state drives the Live / Syncing / Offline header dot (kept from v1.0); on reconnect, queries refetch automatically.
4. Drag-and-drop (dnd-kit) on the board issues PATCH {status, version} optimistically; a 409 conflict rolls the card back and shows "Updated by someone else — refreshed". An undo toast reverses accidental drops.
5. Forms use react-hook-form + zod schemas that mirror the API DTOs, so validation messages appear inline before submission.
6. i18next ships English and Bahasa Indonesia dictionaries; language toggle persisted per user (GAP-9).

5.3 UX requirements carried over and added

- Preserve the validated v1.0 visual language: five-column board, overdue red treatment + banner, urgent pinning, latest-note excerpts on cards, per-member avatar colours, empty states, toasts.
- Add: global ticket search box (header), column sorting, pagination (default 25/page), board column virtualisation beyond 50 cards, per-user "My filters" persistence.
- Accessibility (GAP-10): all interactive elements keyboard-reachable; drag-and-drop has a keyboard alternative (status dropdown remains); WCAG AA contrast; focus traps in dialogs; aria-live for toasts.
- Responsive: board collapses to horizontally scrollable columns on mobile; tables become card lists under 640 px (site engineers are mobile users — screenshot evidence).

5.4 Nginx responsibilities

- Serve the immutable Vite build (content-hashed assets, Cache-Control: max-age=31536000; index.html no-cache).
- TLS termination (TLS 1.2+, HSTS), HTTP→HTTPS redirect.
- Reverse proxy /api/v1 and /ws to the API container; websocket Upgrade handling; request body limit 2 MB; rate limit zone on /api/v1/auth/*.
- Security headers: CSP (self + ws origin), X-Content-Type-Options, Referrer-Policy, frame-ancestors none.

6. Function-by-function production specification

This section re-baselines the 17 functions of PRD v1.0 for the three-tier build. For each: the production flow (frontend → API → database) and the new or changed user stories. The v1.0 positive stories remain valid product behaviour unless amended here; v1.0 stories marked FAILS/GAP are resolved as specified.

6.1 F1 — Application bootstrap (replaces Supabase config screen)

6.1.1 Production flow

1. Browser loads the SPA shell from Nginx; no credentials or workspace identifiers exist in the bundle (env-specific values are limited to the public API origin).
2. The shell calls GET /me with the in-memory token (or silently refreshes via cookie); success routes to the role home, failure routes to /login.
3. GET /workspace supplies branding (company, subtitle) for the header and login screen.
4. The socket connects after authentication and joins the workspace room.

6.1.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F1-P3	Positive	As a returning user with a valid refresh cookie, I want to land directly on my home screen without retyping my password.	Silent refresh issues a new access token; role home renders; no login prompt within the 7-day window.
F1-N4	Negative	As a security reviewer, I must find no secrets in the shipped bundle.	Static analysis of the build output finds no DB URL, no service keys, no workspace UUIDs (closes SEC-1).
F1-N5	Negative	As a user whose refresh token was revoked (password reset), I must be returned to login.	/auth/refresh returns 401; SPA clears state and shows /login with "Session expired".

6.2 F2 — Workspace provisioning (replaces setup wizard)

6.2.1 Production flow

1. Workspace creation becomes an operational task: a seeded CLI script (npm run seed:workspace) creates the workspace, KPI settings row and the first MANAGER account from a config file.
2. All subsequent team building happens in Settings → Users/Sites (F16) by the manager.
3. The v1.0 in-browser wizard is retired (it was dead code in production anyway — BUG-8).

6.2.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F2-P3	Positive	As an operator standing up a new company instance, I want one idempotent seed command so that provisioning is repeatable.	Running the seed twice does not duplicate the workspace; output prints the manager's one-time credentials.
F2-N5	Negative	As an operator, I want provisioning to fail atomically.	Seed runs in one transaction; a failure leaves no partial workspace (closes F2-N4/DATA-3).

6.3 F3 — Authentication & sessions (per-user accounts)

6.3.1 Production flow

1. User enters username + password on /login (role selection removed — the account carries the role).
2. API verifies Argon2id hash, rate-limits failures, issues access JWT + refresh cookie, records last_login in audit.
3. RolesGuard + workspace scoping run on every request; the SPA renders navigation from the role claim.
4. Password change and manager-initiated reset flows as per Section 4.2; deep links (?member=) are retired in favour of real accounts.

6.3.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F3-P4	Positive	As an estimator, I want my own username and password so that my actions and KPI are unambiguously mine.	Login without picking a name from a dropdown; notes and audit entries carry my user id.
F3-N5	Negative	As an attacker brute-forcing the login, I must be throttled and observed.	429 after 5 failures/min; exponential backoff per username; failures logged with IP in audit_log (closes SEC-3 lockout gap).
F3-N6	Negative	As a manager offboarding a user, I want their access to end immediately.	Deactivation revokes all refresh tokens; the next API call after access-token expiry (≤ 15 min) fails; sockets are disconnected on user.updated.
F3-N7	Negative	As a security owner, I require that no password or hash ever reaches a client.	API responses never include password_hash; login errors are uniform ("Invalid username or password") to prevent user enumeration.

6.4 F4 — Ticket creation (Manager / Admin)

6.4.1 Production flow

1. Intake form on /tickets/new validates locally via zod (name, type, priority mandatory — unchanged business rule).
2. POST /api/v1/tickets; the service opens a transaction, increments the workspace ticket sequence, formats ticket_no (EP-NNN), inserts the row with submitted_by_id = the authenticated user, writes the audit entry, commits, then emits ticket.created.
3. The SPA shows the toast with the returned ticket number and clears the form; all connected boards receive the new card within ~ 1 s.
4. Managers can now also create tickets (v1.0 restricted intake to Admins; resolved as Open Question Q2 $\rightarrow$ yes).

6.4.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F4-P3	Positive	As an admin, I want the ticket number the server assigned shown immediately so that I can quote it back to the requester on WhatsApp.	Response contains ticket_no; toast "Ticket EP-119 created".
F4-N4	Negative	As one of two admins submitting simultaneously, we must never receive the same number.	Load test: 50 parallel POSTs produce 50 unique consecutive numbers (closes DATA-1).
F4-N5	Negative	As an admin pasting a 5,000-character description, I want a clear limit error.	400 with field-level message; form preserves input; limit 5,000 chars documented in the DTO.

6.5 F5 — Ticket creation (Site Admin)

6.5.1 Production flow

1. Same form minus the assignee control; the API derives site_id and submitted_by_id from the JWT — the client cannot spoof another site (the server ignores any client-sent site value).
2. Created tickets are unassigned NEW, visible instantly to managers/admins via ticket.created.

6.5.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F5-P3	Positive	As a site engineer, I want my submissions stamped with my site automatically so that scoping cannot go wrong.	site_id set server-side from my account; ticket appears in /site/my-tickets and in the central board.
F5-N3	Negative	As a malicious client crafting a POST with another site's id, I must be ignored or rejected.	API test: site_id in the request body is discarded; resulting ticket carries the JWT's site (closes the SEC-2 exposure for this flow).

6.6 F6 — Dashboard

6.6.1 Production flow

1. GET /tickets/stats returns all aggregates in one call (totals, completion rate, overdue, unassigned, by-status, by-type, per-estimator active/done workload).
2. Aggregation runs in SQL with the shared overdue predicate; ESTIMATOR role may call it for the team board header, SITE_ADMIN receives site-scoped stats.
3. The dashboard subscribes to ticket.* events and refetches stats with a 2 s debounce.

6.6.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F6-P3	Positive	As a manager, I want dashboard numbers that always match the board and table so that I can trust one source.	Stats and list endpoints share SQL predicates; automated test asserts equality across endpoints for a fixture set.

ID	Type	User story	Acceptance criteria
F6-N3	Negative	As a manager on a 5,000-ticket workspace, I want the dashboard to stay fast.	Stats endpoint P95 < 300 ms at 5,000 tickets (indexed aggregates), verified by load test.

6.7 F7 — Kanban board

6.7.1 Production flow

1. GET /tickets?status=... feeds five columns (one query, grouped client-side; server caps at 200 cards per column with "show more").
2. Filters (member incl. Unassigned, status, type) map to query parameters; filter state persists per user in localStorage.
3. Drag-and-drop between columns issues PATCH {status, version}; optimistic move, rollback on 409/403 with explanatory toast; keyboard alternative preserved via the detail page.
4. ticket.updated events move/refresh affected cards live for all viewers.
5. Estimators can drag only their own cards (server enforces; UI disables others).

6.7.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F7-P4	Positive	As an admin, I want to drag a card from New to In Progress Estimation so that triage is one gesture.	Card moves instantly; PATCH succeeds; other viewers see the move within ~1 s; undo toast available for 5 s.
F7-N3	Negative	As an estimator, I must not be able to drag a colleague's card.	Card is not draggable in UI; forged PATCH returns 403 (server test).
F7-N4	Negative	As two users moving the same card at once, the loser must find out immediately.	Second PATCH gets 409; card snaps back with "Updated by someone else"; board refetches.

6.8 F8 — Ticket list

6.8.1 Production flow

1. GET /tickets with page/pageSize/sort/search; server-side trigram search over name, ticket_no, requestor (closes GAP-1).
2. Column headers toggle sort; pagination footer shows total; filters shared with the board.
3. Row click opens /tickets/:id; an export button (Phase 2) downloads the current filter as CSV.

6.8.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F8-P3	Positive	As a manager, I want to type "sodium tank" and see matching tickets in under a second so that lookup stops being a scroll hunt.	Search returns ranked matches P95 < 300 ms at 5,000 tickets.

ID	Type	User story	Acceptance criteria
F8-N3	Negative	As a user paging beyond the last page, I want a sane response.	Empty items array with correct total; UI clamps the page.

6.9 F9 — Ticket detail & update

6.9.1 Production flow

1. GET /tickets/:id returns the ticket, its permission envelope (which fields THIS user may edit — computed server-side and mirrored by the UI), notes (newest first, paged) and KPI award status.
2. The edit form exposes only permitted fields (RBAC matrix, Section 4.3); Save sends PATCH with version.
3. A DONE transition runs the KPI award inside the same DB transaction as the update; leaving DONE inserts a reversal; completed_at is set/cleared server-side.
4. Note submission is a separate POST so a note never silently fails inside a field update; the read-only view shows a correct "Only <assignee name> can update this ticket" message (fixes BUG-1).

6.9.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F9-P4	Positive	As a manager, I want the KPI award to be part of the same save so that Done and points can never diverge.	Transaction test: forcing the KPI insert to fail rolls back the status change; API returns 500 and the ticket stays In Progress (closes BUG-5).
F9-N5	Negative	As an estimator, a forged PATCH on someone else's ticket must fail server-side.	403 from the API regardless of UI state (closes the cosmetic-permissions problem).
F9-N6	Negative	As a user saving stale data, I must not silently overwrite a colleague.	Version mismatch → 409; UI reloads the ticket and re-applies my unsaved note text (closes DATA-6).

6.10 F10 — Ticket deletion (soft)

6.10.1 Production flow

1. DELETE /tickets/:id sets deleted_at, writes an audit entry with the full before-image, emits ticket.deleted.
2. Deleted tickets vanish from all lists/boards/stats but remain queryable in Settings → Audit; a manager can restore within the UI (PATCH deleted_at = null).
3. Ticket numbers are never reused (sequence-based), so restoration is always collision-free.

6.10.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F10-P3	Positive	As a manager who deleted the wrong ticket, I want to restore it with its notes and history intact.	Restore from the audit view brings the ticket back unchanged; KPI ledger unaffected throughout.

ID	Type	User story	Acceptance criteria
F10-N3	Negative	As a compliance owner, I want every deletion attributable.	Audit row records actor, timestamp, IP and before-image (closes GAP-8 for deletions).

6.11 F11 — Estimator "My tickets"

6.11.1 Production flow

1. GET /tickets?assigneeId=me&sort=priority feeds the personal queue; urgent-active pinning and the overdue banner logic are kept exactly as validated in v1.0.
2. ticket.updated events for my assignments update the queue live; a browser notification (permission-gated) fires on new assignment (early GAP-4 slice).

6.11.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F11-P3	Positive	As an estimator, I want to be nudged when new work lands on me so that I do not discover it hours later.	In-app toast + optional browser notification on ticket.updated where assignee becomes me.
F11-N3	Negative	As an estimator, my queue must survive my own rename by the manager.	FK linkage: rename changes display name only; queue and KPI history unaffected (closes DATA-2 for this flow).

6.12 F12 — Site Admin "My tickets" (repaired)

6.12.1 Production flow

1. GET /tickets (server force-scoped to the JWT site) renders the site list with stats.
2. Rows are clickable (fixes BUG-2) → detail page in site mode: read-only metadata + notes, editable required-by date, delete allowed only while status = NEW.
3. Deadline PATCH is field-restricted server-side (a site admin sending status/assignee changes gets 403).

6.12.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F12-P3	Positive	As a site engineer, I want to actually open my ticket and extend the date — the thing v1.0 promised but did not wire up.	Row click opens detail; date change saves and is audited; estimation team sees the new deadline live.
F12-N3	Negative	As a site engineer, deleting a request the team already started must be blocked with guidance.	403/409 with message "Work has started — add a note or ask the estimation team to put it on Hold instead."

6.13 F13 — KPI auto-scoring (server-side)

6.13.1 Production flow

1. TicketsService.update detects the DONE transition and calls KpiService.award within the same transaction.
2. Scoring: completed_at (server clock, workspace timezone) ≤ deadline end-of-day → point_on_target; after → point_miss_target; no deadline → point_on_target but the entry is tagged "no deadline set" for manager visibility (partial answer to v1.0 design note F13-N4; policy decision Q4 remains open).
3. Uniqueness by partial index; award month = completion month (confirmed Q3 → completion month).
4. Leaving DONE inserts a REVERSAL entry; a subsequent completion re-awards fresh points against the then-current deadline.

6.13.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F13-P3	Positive	As a manager, I want reopened-then-late tickets rescored honestly so that the ledger reflects reality.	Reopen writes -3 reversal; late re-completion writes -2 miss entry; ledger history shows all three entries.
F13-N5	Negative	As an estimator, a KPI write failure must never be silent.	Award failure rolls back the whole save and surfaces an error; no Done-without-points state can exist (closes BUG-5, F13-N2).
F13-N6	Negative	As a manager, deadline-less completions must be visible for review.	KPI summary flags "no-deadline awards" count per member per month.

6.14 F14 — KPI dashboard (Manager)

6.14.1 Production flow

1. GET /kpi/summary renders the month strip and leaderboard (server-aggregated); year selector offers every year with data.
2. Settings PUT, bulk opening POST (single transaction, upsert per member) and manual entry POST as per API catalogue; manual entries require a description (tightened from v1.0 where it was optional).
3. CSV/XLSX export per year; manual-entry badge styled distinctly (fixes BUG-3).

6.14.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F14-P4	Positive	As a manager, I want to export the year ledger to Excel so that KPI feeds the HR review cycle.	Export contains every entry with member, month, type, points, description, ticket reference.
F14-N5	Negative	As a manager granting opening points twice for a month, I must not duplicate allowances.	Upsert semantics + unique index: re-submission replaces, never duplicates (hardens v1.0 delete-then-insert).

6.15 F15 — KPI view (Estimator)

6.15.1 Production flow

1. Same summary/entries endpoints self-scoped; year selector across all history (closes GAP-7).

2. `kpi.updated` events refresh the view live when the manager posts a bonus or an award lands.

6.15.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F15-P3	Positive	As an estimator in January, I want to review my full previous-year record for my appraisal.	Year selector lists all years with entries; totals match the manager export.

6.16 F16 — Settings & administration

6.16.1 Production flow

1. Users tab: create/edit/deactivate users, assign roles and sites, trigger password resets (temporary password shown once, never stored readable — closes SEC-4); duplicate usernames rejected by the DB unique constraint (closes F16-N4).
2. Sites tab: CRUD with delete protection while active tickets exist.
3. Workspace tab: company/subtitle/prefix (prefix change applies to future tickets only).
4. Audit tab: filterable trail of mutations incl. deletions and restores.
5. The v1.0 "new workspace" reset and member deep links are retired (BUG-6/BUG-7).

6.16.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F16-P4	Positive	As a manager, I want to onboard an estimator with a temporary password that they must change on first login.	User created; temp password displayed once; first login forces change; audit records both events.
F16-N5	Negative	As a manager, deactivating a user with 12 open tickets must warn me and offer bulk reassignment.	Deactivation dialog lists open tickets and offers "unassign all" / "reassign to..." before confirming.

6.17 F17 — Realtime synchronisation

6.17.1 Production flow

1. `socket.io` gateway authenticates the JWT on handshake, joins room `ws:<workspaceid>` (site admins additionally `site:<siteid>` for scoped events).
2. Every committed mutation emits an event with entity ids and changed fields; TanStack Query invalidates matching keys; UI updates in ~1 s.
3. Heartbeat + automatic reconnect with exponential backoff; on reconnect, active queries refetch (no stale board after sleep/wake).
4. Header dot: green connected, amber reconnecting, red disconnected — semantics preserved from v1.0.

6.17.2 New / changed user stories

ID	Type	User story	Acceptance criteria
F17-P3	Positive	As a manager in a stand-up, I want the board to reflect a colleague's change within a second so that we stop refreshing.	Median propagation < 1 s, P95 < 3 s (measured).
F17-N3	Negative	As a user behind a proxy that blocks websockets, I still need a working app.	socket.io falls back to HTTP long-polling through the same Nginx route; feature parity retained.

7. Security requirements (production)

#	Requirement
SR-1	Passwords hashed with Argon2id (memory 64 MB, iterations tuned to ~100 ms); minimum length 10; common-password denylist; no maximum-length truncation below 128.
SR-2	JWT access tokens ≤ 15 min lifetime, HS512 or RS256; refresh tokens 7 days, httpOnly Secure SameSite=Strict cookies, rotated on every use, revocable server-side; replayed refresh tokens revoke the family.
SR-3	Every endpoint behind JwtAuthGuard except /auth/login, /auth/refresh, /health, /ready; RolesGuard + workspace/site scoping at service level; deny by default.
SR-4	Rate limiting: 5 login attempts/min/IP with per-username backoff; 100 req/min/user on the general API; enforced at Nginx and Nest (throttler).
SR-5	Input handling: DTO validation with whitelisting; parameterised queries only (Prisma); output encoding by React JSX; no dangerouslySetInnerHTML anywhere (CI lint rule).
SR-6	Transport: TLS 1.2+ only, HSTS 1 year; DB connections on a private network, credentials rotated on personnel change.
SR-7	Audit: all auth events (success/failure/reset) and all mutations logged with actor, IP, timestamp; logs retained ≥ 12 months.
SR-8	Backups: nightly pg_dump, 30-day retention, quarterly restore drill; RPO ≤ 24 h, RTO ≤ 4 h.
SR-9	Dependency hygiene: npm audit + Renovate in CI; images rebuilt monthly; no secrets in Git (gitleaks in CI).
SR-10	Pre-launch verification: OWASP Top-10 checklist review + an external or internal penetration test of auth, RBAC and IDOR paths; all Critical/High findings closed before go-live.

8. Non-functional requirements (measurable)

#	Category	Target
NFR-1	Performance	API P95 < 300 ms for list/stats at 5,000 tickets, 30 concurrent users; SPA first meaningful paint < 2 s on 4G; board interaction < 100 ms local response.
NFR-2	Realtime	Mutation-to-peer-visibility median < 1 s, P95 < 3 s.
NFR-3	Availability	99.5% monthly; graceful degradation to long-polling; health probes with alerting.
NFR-4	Capacity	Designed for 50 concurrent users, 50,000 tickets, 500,000 notes without architecture change.
NFR-5	Compatibility	Last 2 versions of Chrome/Edge/Safari/Firefox; iOS and Android mobile browsers (site engineers).
NFR-6	Accessibility	WCAG 2.1 AA on all screens; keyboard-complete flows.
NFR-7	Localisation	EN + ID at launch; all user-facing strings externalised.
NFR-8	Data integrity	Zero duplicate ticket numbers under load test; KPI ledger reconciles to ticket history exactly in automated audit job.

9. Deployment & operations

9.1 Docker Compose topology

```
services:
  nginx:    ports 80/443; volumes: spa build, certs; depends_on: api
  api:      NestJS (node:20-alpine, multi-stage build); env from .env; depends_on: db
  (healthy)
  db:       postgres:16-alpine; named volume; healthcheck pg_isready
  backup:   cron pg_dump -> /backups (30-day rotation)
```

9.2 CI/CD pipeline (GitHub Actions)

1. On pull request: lint (eslint + no-dangerouslySetInnerHTML rule), typecheck, unit tests, API e2e tests against a disposable Postgres service, FE component tests, gitleaks, npm audit.
2. On merge to main: build FE bundle and API image, tag with git SHA, push to registry.
3. Deploy to staging automatically; run Prisma migrations (migrate deploy) as a pre-start step; smoke tests hit /ready and a login flow.
4. Production deploy is a manual approval of the same artefacts; migrations run before traffic switch; rollback = redeploy previous tag (migrations are forward-only and backward-compatible for one release).

9.3 Monitoring

- Structured JSON logs (pino) shipped to a log store; error alerting on 5xx rate and failed-login spikes.
- Metrics: request latency histograms, socket connection count, DB pool saturation, ticket-creation rate.
- Uptime checks on /health (liveness) and /ready (DB connectivity).

10. Testing strategy

Layer	Scope	Tooling / gate
Unit (API)	Ticket numbering, KPI award/reversal math, RBAC field-permission envelope, overdue predicate, DTO validation.	Jest; $\geq 85\%$ coverage on services; CI-blocking.
Integration (API)	Every endpoint $\times$ role matrix (expected 200/403 per Section 4.3); optimistic-lock conflicts; concurrency tests (parallel ticket creation, parallel DONE saves).	Jest + supertest against real Postgres in CI.
Frontend	Form validation, board drag interactions (incl. rollback), permission-driven rendering, i18n snapshots.	Vitest + React Testing Library; Playwright for the 6 golden paths.
End-to-end (golden paths)	Login $\rightarrow$ create $\rightarrow$ assign $\rightarrow$ note $\rightarrow$ done $\rightarrow$ KPI visible; site admin submit $\rightarrow$ track $\rightarrow$ extend date; manager KPI month cycle; offboarding; conflict handling; restore deleted ticket.	Playwright on staging; release-blocking.
Migration	Legacy CSV $\rightarrow$ new schema on staging: row counts, KPI sums, unmatched-name report empty.	Scripted validation gate before cutover.

Layer	Scope	Tooling / gate
Load	NFR-1/NFR-2 targets at 5,000 tickets / 30 users; duplicate-number check under 50-way parallel create.	k6; pre-launch.
Security	Section 7 SR-10 checklist + pen test.	Pre-launch, findings triaged.
UAT	Scenario scripts derived from every user story in Section 6 and the retained v1.0 stories.	KPN Downstream pilot group sign-off.

11. Delivery plan

Milestone	Contents	Exit criteria	Indicative duration
M1 — Foundations	Repo + CI/CD, Docker skeleton, schema + migrations, AuthModule, UsersModule, app shell with login.	Login/refresh/RBAC e2e green; deployed to staging.	2–3 weeks
M2 — Ticket core	Tickets/Notes modules, numbering, board, list, detail, dashboard, soft delete + audit.	Golden paths 1–2 pass; concurrency tests green.	3–4 weeks
M3 — KPI	KPI modules (auto/reversal/opening/manual/summary/export), manager + member KPI screens.	Ledger reconciliation job green on fixtures.	2 weeks
M4 — Realtime & UX polish	Events gateway, live board, drag-and-drop, search/pagination, i18n, accessibility pass.	NFR-2 measured; axe audit clean.	2 weeks
M5 — Migration & go-live	Migration scripts + rehearsal, UAT, load + security tests, cutover, hypercare.	UAT sign-off; legacy Supabase set read-only then decommissioned.	2 weeks + 2 weeks hypercare

12. Traceability — v1.0 findings resolved

v1.0 finding	Resolution in v2.0	Section
SEC-1 Hard-coded credentials	No secrets in client; env-based server config	2.3, 6.1
SEC-2 allow_all database	DB private to API; server RBAC everywhere	2.1, 4.3
SEC-3 Plain-text shared passwords	Per-user accounts, Argon2id, rate limiting, lockout-by-backoff	4.2, 7
SEC-4 Passwords displayed in UI	Reset-only flows; one-time temp passwords	6.16
SEC-5 Stored XSS	React escaping + DTO validation + CSP + lint gate	5, 7 SR-5
SEC-6 No session management	JWT + rotating refresh cookies, revocation	4.2
DATA-1 Ticket number races/reuse	Atomic per-workspace sequence	3.3, 6.4
DATA-2 Name-string linkage	Foreign keys throughout; renames are metadata	3.2, 6.11
DATA-3 Non-transactional writes	Service-level transactions (Prisma \$transaction)	3.3, 6.9
DATA-4 KPI double-award race	Partial unique index in engine	3.3, 6.13
DATA-5 Schema drift	Versioned migrations, single source of truth	3.1
DATA-6 Client clocks / lost updates	Server time authoritative; optimistic locking (409)	3.3, 6.9
BUG-1 Literal template text	Rebuilt detail view with server permission envelope	6.9
BUG-2 Site rows not clickable	Site detail page wired; deadline edit works	6.12

v1.0 finding	Resolution in v2.0	Section
BUG-3 Undefined badge class	Design-system badge components	6.14
BUG-5 Silent KPI failure	Award inside the save transaction	6.13
BUG-6/7/8 Dead code paths	Wizard/config/deep links retired; CLI provisioning	6.1, 6.2, 6.16
GAP-1 No search	Trigram search endpoint + header search	6.8
GAP-2 Polling & scale ceilings	WebSocket events + pagination	6.17, 6.8
GAP-3 No drag-and-drop	dnd-kit board with undo + conflict rollback	6.7
GAP-4 No notifications	Browser notification slice now; e-mail/WA Phase 2	6.11, 4.1
GAP-5 No attachments	Phase 2 (object storage) — out of v2.0 scope	13
GAP-6 No KPI reversal/export	REVERSAL entries + CSV/XLSX export	6.13, 6.14
GAP-7 KPI year limits	Full history, any year	6.14, 6.15
GAP-8 No audit / hard delete	Audit log + soft delete + restore	6.10, 4.1
GAP-9 English only	i18next EN + ID	5.3
GAP-10 Accessibility	WCAG 2.1 AA requirement + axe gate	5.3, 10

13. Scope decisions and open questions

- DECIDED — Backend framework: NestJS (dedicated API) rather than Next.js, because the frontend is a standalone React SPA behind Nginx (Section 1.1).
- DECIDED — Managers can create tickets (v1.0 Q2).
- DECIDED — KPI month attribution follows completion month (v1.0 Q3), now with server-authoritative time.
- DECIDED — Site Admin deletion narrowed to NEW tickets only (behaviour change vs v1.0 — communicate in UAT).
- OPEN Q1 — tender_notes: obtain the deployed EngPro source to confirm this feature's schema and UI before M2 freeze; otherwise migrate the table verbatim and defer the UI.
- OPEN Q2 — Deadline-less tickets: keep allowed-with-flag (current spec) or make deadline mandatory at intake? Recommendation: mandatory for Project/Ops Tender types, optional otherwise.
- OPEN Q3 — Hosting target: customer VPS vs cloud VM; affects backup and TLS automation details only.
- OPEN Q4 — Attachments (GAP-5) sizing and storage budget for Phase 2.
- DEFERRED to Phase 2: e-mail/WhatsApp notifications, attachments, CSV ticket export, per-user saved filters sync, reporting dashboards.